

TO DO LIST WEB PAGE

AIM:

To design and develop a Dynamic To-Do List Web Application that allows users to add, edit, delete, and mark tasks as completed using HTML, CSS, and JavaScript.

TASKS:

- . Created project folder dynamic _to do _list
- . Designed To-Do interface using HTML
- . Implemented dynamic task addition using JavaScript

TECH STACK USED:

HTML,CSS ,JAVASCRIPT, JSON

CODE:

Frontend:

Index.html, style.css . script.js

```
index.html X
frontend > index.html > html > body > div.container
1 <!DOCTYPE html>
2 <html>
3 <head>
4 <title>My To-Do List</title>
5 <link rel="stylesheet" href="style.css">
6 </head>
7 <body>
8
9 <div class="container">
10 <h1>My To-Do List</h1>
11
12 <input type="text" id="title" placeholder="Task Title">
13 <textarea id="desc" placeholder="Description"></div>
14
15 <select id="priority">
16 <option value="Low">Low</option>
17 <option value="Medium" selected="true">Medium</option>
18 <option value="High">High</option>
19 </select>
20
21 <button onclick="addTask()">Add Task</button>
22
23 <div class="filters">
24 <button onclick="loadTasks()">All</button>
25 <button onclick="filterTasks('active')">Active</button>
26 <button onclick="filterTasks('completed')">Completed</button>
27 </div>
28
29 <h3 id="counter"></h3>
30
31 <ul id="tasklist"></ul>
32 </div>
33
34 <script src="script.js"></script>
35 </body>
36 </html>

# style.css X
frontend > # style.css > body
1 body{
2   font-family: Arial;
3   background-color: #f4f4f4;
4   display: flex;
5   justify-content: center;
6 }
7
8 .container{
9   width: 400px;
10  background-color: white;
11  padding: 20px;
12  margin-top: 40px;
13  border-radius: 10px;
14  box-shadow: 0 0 10px gray;
15 }
16
17 input,textarea,select,button{
18   width: 100%;
19   margin-top: 10px;
20   padding: 8px;
21 }
22
23 button{
24   background-color: #007bff;
25   color: white;
26   border: none;
27   cursor: pointer;
28 }
29
30 button:hover{
31   background-color: #0056b3;
32 }
33
34 .completed{
35   text-decoration: line-through;
36   color: gray;
37 }
38
39 .badge-low{color: green;}
40 .badge-medium{color: orange;}
41 .badge-high{color: red;}
42
43 .filters button{

JS script.js X
frontend > JS script.js > ...
1 const API="http://localhost:3000/tasks";
2
3 async function loadTasks(){
4   let res=await fetch(API);
5   let data=await res.json();
6   showTasks(data);
7 }
8
9 async function addTask(){
10  let title=document.getElementById("title").value
11  let priority=document.getElementById("priority").value
12  if(title=="" || title.length>50) return;
13
14  await fetch(API,{
15    method:"POST",
16    headers:{"Content-Type":"application/json"},
17    body:JSON.stringify({title,priority})
18  });
19
20  document.getElementById("title").value="";
21  loadTasks();
22 }
23
24 function showTasks(tasks){
25  let list=document.getElementById("tasklist");
26  list.innerHTML="";
27
28  let completed=0;
29
30  tasks.forEach(t=>{
31    if(t.isDone) completed++;
32
33    let li=document.createElement("li");
34    li.className=t.isDone?"completed":"";
35
36    li.innerHTML=`
37    <input type="checkbox" ${t.isDone?"checked":""}
38    ${t.title}
39    <span class="badge-${t.priority.toLowerCase()}>
40    <button onclick="editTask(${t.id})">Edit</button>
41    <button onclick="del(${t.id})">Delete</button>
42    `;
43  }
```

Backend:

Package. Json, server.js

```
() package.json X
backend > () package.json > ...
1 {
2   "name": "todo",
3   "version": "1.0.0",
4   "main": "server.js",
5   "dependencies": {
6     "express": "^4.18.2",
7     "sqlite3": "^5.1.6",
8     "cors": "^2.8.5"
9   }
10 }

JS server.js X
backend > JS server.js > ...
1 const express=require("express");
2 const sqlite3=require("sqlite3").verbose();
3 const cors=require("cors");
4
5 const app=express();
6 app.use(cors());
7 app.use(express.json());
8
9 const db=new sqlite3.Database("db.sqlite");
10
11 db.run('CREATE TABLE IF NOT EXISTS tasks(
12   id INTEGER PRIMARY KEY AUTOINCREMENT,
13   title TEXT NOT NULL,
14   priority TEXT DEFAULT "Medium",
15   isDone INTEGER DEFAULT 0,
16   createdAt DATETIME DEFAULT CURRENT_TIMESTAMP
17 );');
18
19 app.get("/tasks",(req,res)=>{
20   db.all("SELECT * FROM tasks ORDER BY id DESC",[],(e,r)=>res.json(r));
21 });
22
23 app.post("/tasks",(req,res)=>{
24   let {title,priority}=req.body;
25   db.run("INSERT INTO tasks(title,priority) VALUES(?,?)",[title,priority]);
26   res.json({id:this.lastID});
27 });
28
29
30 app.put("/tasks/:id",(req,res)=>{
31   let {title,priority}=req.body;
32   db.run("UPDATE tasks SET title=?,priority=? WHERE id=?",[title,priority]);
33 });
34
35 app.patch("/tasks/:id/status",(req,res)=>{
36   db.run("UPDATE tasks SET isDone = CASE WHEN isDone=0 THEN 1 ELSE 0");
37 });
```

OUTPUT:

